

Dokumentasi Source Code – Tugas 5: Manajemen Produk

Sumber: tugas-5/index.html

1. Struktur Halaman dan Kontrol HTML (baris 8–26)

```
<h1>Manajemen Produk</h1>

<div id="daftarProduk"></div>

<hr>

<h2>Tambah Produk</h2>
<input type="number" id="inputId" placeholder="ID" value="6">
<input type="text" id="inputNama" placeholder="Nama" value="Tablet">
<input type="number" id="inputHarga" placeholder="Harga" value="4000000">
<button id="btnTambah">Tambah Produk</button>

<h2>Hapus Produk</h2>
<input type="number" id="inputHapusId" placeholder="ID yang dihapus" value="2">
<button id="btnHapus">Hapus Produk</button>

<hr>

<button id="btnTampilkan">Tampilkan Semua Produk</button>
```

Bagian ini menyediakan tampilan sederhana untuk aplikasi: judul, elemen **<div id="daftarProduk">** sebagai tempat menampilkan daftar produk, input untuk data produk baru, input ID yang akan dihapus, serta tiga tombol (**btnTambah**, **btnHapus**, **btnTampilkan**) yang nantinya dijalankan melalui event listener.

2. List Data Produk Awal (baris 29–36)

```
// Daftar produk awal
let produkList = [
  { id: 1, nama: "Laptop", harga: 12000000 },
  { id: 2, nama: "Smartphone", harga: 5000000 },
  { id: 3, nama: "Keyboard", harga: 350000 },
  { id: 4, nama: "Mouse", harga: 200000 },
  { id: 5, nama: "Monitor", harga: 1800000 }
];
```

Array **produkList** menyimpan daftar produk awal sebanyak lima buah. Setiap produk berupa object dengan property **id**, **nama**, dan **harga**. Array ini dideklarasikan dengan **let** karena isinya akan diganti saat produk ditambah atau dihapus.

3. Menambahkan Produk – Spread Operator (baris 38–42)

```
// Menambahkan produk baru. Spread operator dipakai untuk
// menyusun array baru dari produkList lama ditambah produk baru.
function tambahProduk(id, nama, harga) {
  produkList = [...produkList, { id, nama, harga }];
}
```

Function **tambahProduk(id, nama, harga)** menambahkan produk baru ke dalam array. Spread operator (**...produkList**) dipakai untuk menyalin seluruh produk lama ke array baru, lalu object produk baru ditambahkan di belakangnya. Penulisan **{ id, nama, harga }** adalah cara singkat membuat object dari parameter.

4. Menghapus Produk – Rest Parameter (baris 44–48)

```
// Menghapus produk berdasarkan ID. Rest parameter dipakai untuk
// menampung semua ID yang dikirim ke dalam satu array bernama ids.
function hapusProduk(...ids) {
  produkList = produkList.filter((produk) => !ids.includes(produk.id));
}
```

Function **hapusProduk(...ids)** memakai rest parameter (**...ids**) sehingga semua ID yang dikirim sebagai argument otomatis dikumpulkan menjadi satu array. Produk yang ID-nya ada di dalam array tersebut dibuang menggunakan **filter()** dan **includes()**.

5. Menampilkan Produk – Destructuring (baris 50–58)

```
// Menampilkan seluruh produk. Destructuring dipakai untuk
// mengambil id, nama, dan harga dari setiap object produk.
function tampilkanProduk() {
  const daftar = produkList.map((produk) => {
    const { id, nama, harga } = produk;
    return `<li>ID: ${id} | Nama: ${nama} | Harga: Rp ${harga.toLocaleString("id-ID")}</li>`;
  });
  document.getElementById("daftarProduk").innerHTML = "<ul>" + daftar.join("") + "</ul>";
}
```

Function **tampilkanProduk()** menampilkan seluruh isi **produkList**. Destructuring (**const { id, nama, harga } = produk**) dipakai untuk mengambil ketiga property dari setiap object produk, lalu hasilnya disusun menjadi daftar **** dan ditulis ke halaman melalui **innerHTML**.

6. Event Listener (baris 60–80)

```
// Event listener untuk menampilkan produk
document.getElementById("btnTampilkan").addEventListener("click", tampilkanProduk);

// Event listener untuk menambah produk
document.getElementById("btnTambah").addEventListener("click", () => {
  const id = Number(document.getElementById("inputId").value);
  const nama = document.getElementById("inputNama").value;
  const harga = Number(document.getElementById("inputHarga").value);
  tambahProduk(id, nama, harga);
  tampilkanProduk();
});

// Event listener untuk menghapus produk
document.getElementById("btnHapus").addEventListener("click", () => {
  const id = Number(document.getElementById("inputHapusId").value);
  hapusProduk(id);
  tampilkanProduk();
});

// Tampilkan daftar produk awal saat halaman dibuka
tampilkanProduk();
```

Bagian ini menghubungkan tombol dengan fungsinya memakai **addEventListener("click", ...)**. Tombol **btnTampilkan** memanggil **tampilkanProduk()**, tombol **btnTambah** membaca input lalu memanggil **tambahProduk()**, dan tombol **btnHapus** membaca ID lalu memanggil **hapusProduk()**. Baris terakhir memanggil **tampilkanProduk()** sekali agar daftar awal langsung terlihat saat halaman dibuka.